Task 1 – 8-Puzzle with A* Search

Documentation

Introduction to Artificial Intelligence and Data Science, WS 26/27
Group members: Dominik Barat, Daniel Rosenthal, Fabian Sefranek, Benjamin Suskic

1. Task description

The objective of this task is to implement an A* search algorithm in Python to solve the 8-puzzle and compare the performance of two heuristic functions: Hamming distance and Manhattan distance.

The 8-puzzle consists of a 3×3 board containing eight numbered tiles and one blank space. A move consists of sliding a tile adjacent to the blank into the empty position, with each move having a cost of 1. The goal is to reach the ordered state

```
0 1 2
3 4 5
6 7 8
```

using as few moves as possible.

The implementation generates random start states, checks their solvability, and solves them using A* with both heuristic functions. To evaluate their performance, the same 100 solvable start states are tested with each heuristic. The number of expanded nodes, execution time, and solution length are recorded. Their mean and standard deviation are calculated to compare the results.

Finally, the documentation discusses the time and space complexity of the heuristics and compares their performance based on the experimental results.

How to run it

The script needs only the Python standard library (Python 3.8 or newer).

Command	Effect
`python puzzle8.py`	Runs the experiment with 100 random solvable puzzles using seed 42 and displays an example solution.

2. Software architecture

The program is organized into seven logical components (modules), each responsible for a specific part of the solution process. The Main component coordinates these components and controls the overall execution.

First, the puzzle model generates the goal state and random starting states. Each starting state is checked for solvability before it is included in the experiment. The experiment then solves each valid starting state twice: once using the Hamming heuristic and once using the Manhattan heuristic.

During each search, A* explores possible puzzle states and uses the selected heuristic to estimate how far each state is from the goal. The algorithm returns a solution path and the number of expanded nodes. The experiment also measures the execution time and solution length of each search.

Finally, the statistics component summarizes the collected measurements by calculating the mean and standard deviation for each heuristic. The results are printed in a report in the console.

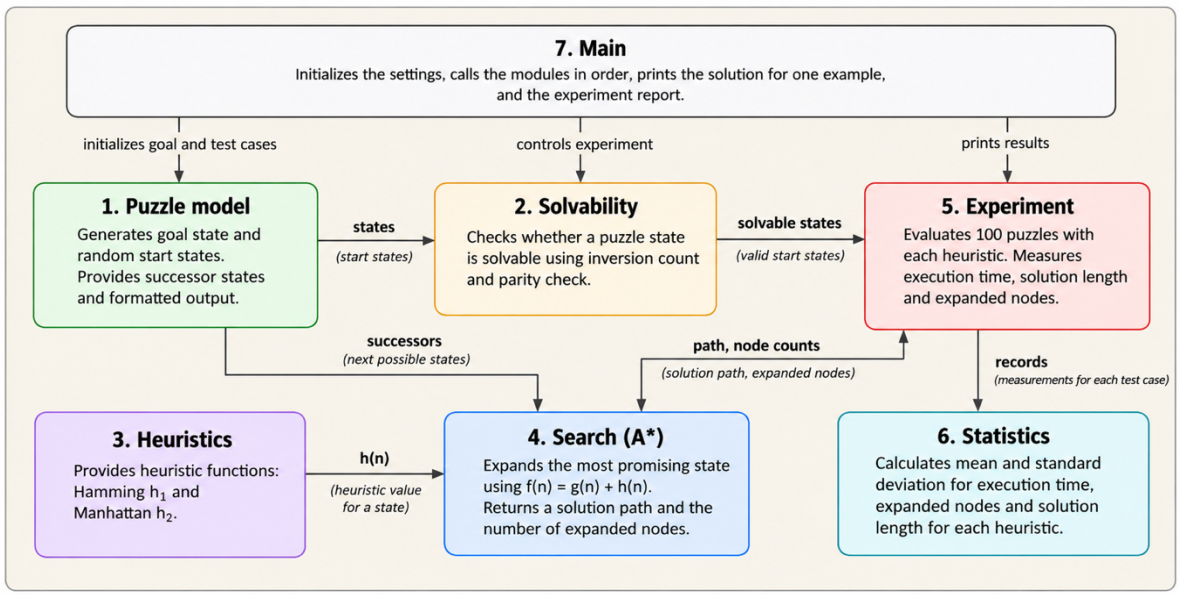


Figure 1: Components of puzzle8.py and the data passed between them.

3. Modules and interfaces

The implementation is contained in a single Python file, puzzle8.py, organized into seven logical components. A puzzle state is represented as a tuple of nine integers with 0 representing the blank space.

Component	Responsibility	Input	Output
1. Puzzle model	Defines the goal state, generates random states and legal successors, and formats puzzle states.	Puzzle state, random generator	Goal state, start states, successor states, formatted output

Component	Responsibility	Input	Output
2. Solvability	Decides whether the goal is reachable from a given start state.	Start state, goal state	Boolean (True or False)
3. Heuristics	Estimates the remaining distance to the goal using Hamming or Manhattan distance.	Puzzle state, goal state	Integer estimate h(n)
4. Search (A*)	Finds an optimal solution using the selected heuristic.	Start state, goal state, heuristic function	Solution path, number of expanded nodes
5. Experiment	Generates solvable test cases, runs both heuristics on the same start states, and measures performance.	Number of puzzles, goal state, random generator, heuristic functions	Individual measurements for each run
6. Statistics	Summarizes the collected measurements.	Experiment records	Mean and standard deviation for each measure and heuristic
7. Main	Initializes the settings, coordinates execution, displays an example solution, and prints the results.	Configuration settings	Console output

The components interact through function calls. For example, the A* search receives a heuristic function as a parameter, allowing both heuristics to use the same search implementation. The experiment collects the search results, which are then processed by the statistics component.

Example output of a solution

For the example state, the script prints the solution step by step, showing the moved tile and the resulting board after each move. The example solution is generated using the fixed random seed 42. The first two and the last of 23 steps are shown below:

```
Start:
2 5 7
8 6 1
4 _ 3

Step 1 - move tile 6
2 5 7
8 _ 1
4 6 3

Step 2 - move tile 8
2 5 7
_ 8 1
4 6 3
...
Step 23 - move tile 1
_ 1 2
3 4 5
6 7 8

Goal reached in 23 moves
```

4. Design decisions

The implementation was designed to make the search efficient, the comparison between the two heuristics fair, and the results easy to verify. The following sections explain the main design decisions and why they were made.

1. Representing puzzle states as tuples.

Each puzzle state is stored as a tuple of nine integers instead of a two-dimensional list. For example, the goal state is represented as (0, 1, 2, 3, 4, 5, 6, 7, 8). The tuple represents the board row by row, with 0 representing the blank space. Tuples cannot be changed after creation and can be used as keys in dictionaries or stored in sets. This makes it easier for A* to recognize states that have already been encountered and avoid unnecessary repeated work.

2. Checking solvability before searching.

Not every possible arrangement of the puzzle can reach the goal state. The program therefore counts inversions, which occur when a larger numbered tile appears before a smaller one in the row-by-row sequence. For a 3×3 puzzle, the parity (whether the count is even or odd) determines whether the state can reach the configured goal. The program compares the inversion parity of the start state with that of the goal state. This check prevents the search from wasting time on impossible puzzles.

3. Generating random but solvable test cases.

To create test cases, the program randomly shuffles the nine puzzle elements and checks whether each resulting state is solvable. Unsolvable states are discarded until the required number of solvable states has been collected. This approach allows the experiment to use randomly selected solvable states rather than only states created by moving a few steps away from the goal. It also ensures that both heuristics are tested on the same set of puzzles.

4. Using two interchangeable heuristic functions.

The program implements Hamming distance and Manhattan distance to estimate how far a puzzle state is from the goal. Hamming distance counts the numbered tiles that are in the wrong position. Manhattan distance adds up how many horizontal and vertical steps each tile is away from its goal position.

Both heuristics are implemented as separate functions and passed to the same A* algorithm. This means the search algorithm does not need to be rewritten to test a different heuristic. The two functions also allow the program to compare how different estimates affect search performance.

5. Choosing A and avoiding unnecessary repeated searches.*

A* evaluates each state using (f(n) = g(n) + h(n)). Here, (g(n)) is the number of moves made so far, while (h(n)) estimates the number of moves remaining to reach the goal. The algorithm prioritizes states with the lowest estimated total cost.

Both implemented heuristics are admissible, meaning they never estimate a remaining cost higher than the true minimum. Consequently, A* can find an optimal solution with either heuristic. Manhattan distance also dominates Hamming distance because its estimate is always at least as large and is often more informative. This generally helps A* explore fewer states.

To manage the search efficiently, the program uses a priority queue (heapq) to select the next state. Each queue entry contains the estimated total cost, heuristic value, path cost, and puzzle state. If two states have the same estimated total cost, the one with the smaller heuristic value is prioritized.

The program also uses a dictionary to store the best known path cost for each encountered state, another dictionary to keep track of parent states for reconstructing the solution path, and a set to

track states that have already been expanded. These structures help avoid unnecessary repeated expansions and make it possible to reconstruct the solution once the goal is reached.

6. Ensuring a fair and reproducible comparison.

Both heuristics are tested on the same 100 solvable start states. This is important because otherwise differences in difficulty between randomly generated puzzles could influence the results. A fixed random seed makes the sequence of generated states reproducible when the same program and settings are used again.

The program measures execution time using Python's time.perf_counter(). It also records the number of expanded nodes and the solution length for each search. These measurements provide different perspectives on search performance: execution time measures how long a search takes, while the number of expanded nodes indicates how much work the algorithm performs. Solution length is used to verify that both heuristics find equally short solutions.

7. Verifying and presenting the results.

The program includes a self-check that compares the solution lengths returned by the two heuristics for each puzzle. Since both heuristics are admissible, the solutions should have the same optimal length. Any difference triggers a warning and provides a reason to investigate the implementation.

In addition, the program prints the solution to one example puzzle step by step, including the moved tile and the board after each move. This makes it possible to inspect the solution and verify that the goal state is reached. Finally, the program prints the mean and standard deviation of the measured values for both heuristics.

5. Results and complexity

Complexity estimate

The 8-puzzle has 9! = 362,880 possible arrangements, of which half, 181,440, are reachable from a given goal state. The blank has 2, 3 or 4 possible moves, so the average branching factor b is approximately 3.

A* has exponential worst-case time and space complexity, commonly expressed as O(b^d), where b is the branching factor and d is the solution depth. Both heuristics reduce the number of states that need to be explored, but they do not change this worst-case complexity class.

Quantity	Estimate
Total possible arrangements	9! = 362,880
Reachable states	181,440
Average branching factor	Approximately 3
Uninformed tree search to depth 22	About 3^22 ≈ 3.1 × 10^10 nodes
A* time complexity (worst case)	O(b^d)
A* space complexity (worst case)	O(b^d)
Upper bound with duplicate detection	181,440 states, the size of the reachable state space
Solvability check	28 pairwise comparisons, O(1) for the fixed 8-puzzle

Heuristic complexity comparison

Both Hamming and Manhattan distance examine the numbered tiles to calculate their estimates. For the fixed 8-puzzle, each heuristic evaluation requires examining at most eight tiles, making the computational complexity O(8), or O(1) for this fixed board size.

Property	Hamming distance	Manhattan distance
Heuristic evaluation	O(1)	O(1)
Admissible	Yes	Yes
Consistent	Yes	Yes
Dominance	Dominated by Manhattan	Dominates Hamming
Information provided	Counts misplaced tiles	Measures total horizontal and vertical distance
Expected search effort	Higher	Lower

Manhattan distance provides a more informed estimate because it considers how far each tile must move, rather than only whether it is misplaced. Although both heuristics have the same computational complexity per evaluation, Manhattan generally reduces the number of states expanded by A*.

Measured comparison

The following values show the mean and standard deviation over 100 random solvable puzzles (seed 42). Each puzzle was solved once with each heuristic.

Measure	Hamming	Manhattan
Expanded nodes	13,316 ± 12,942	835 ± 831
Run time [ms]	75.97 ± 85.06	3.77 ± 4.46
Solution length [moves]	22.47 ± 3.41	22.47 ± 3.41

To collect 100 solvable puzzles, 92 unsolvable random states were discarded. In all 100 puzzles, both heuristics returned the same solution length.

The execution times were measured using Python's time.perf_counter() on a local Mac computer. They depend on the hardware and system load, whereas the number of expanded nodes remains the same when the program and random seed are unchanged.

Reading the numbers

- **Search effort.** Hamming expands approximately 16 times as many nodes as Manhattan. This shows that Manhattan provides a more informative heuristic and allows A* to explore fewer states.
- **Execution time.** Hamming takes approximately 10 times as long as Manhattan. The difference is mainly explained by the higher number of expanded nodes.

- **Solution quality.** Both heuristics produce the same average solution length of 22.5 moves. This confirms that the choice of heuristic affects search performance but not the optimal solution length.
- **Spread.** The standard deviation is relatively high for both node counts and execution times. This indicates that the computational effort varies considerably depending on the difficulty of the individual puzzle instances.

6. Discussion and conclusions

The experimental results show that the choice of heuristic has a significant impact on the performance of A*. Using the same search implementation, Manhattan expands approximately 16 times fewer nodes and is about 10 times faster than Hamming. Both heuristics find solutions with the same length, but Manhattan requires considerably less search effort.

Conclusions

Both heuristics find optimal solutions. Hamming and Manhattan distance are admissible, so A* returned a shortest solution for all 100 puzzles. The two heuristics differ in search effort rather than solution quality.

Manhattan performs significantly better than Hamming. For every puzzle state, Manhattan distance is at least as large as Hamming distance and is usually more informative because it considers the actual distance of each misplaced tile from its goal position. As a result, A* expands approximately 16 times fewer nodes and requires less execution time.

The experiment also confirms the importance of testing both heuristics on identical starting states. This makes the comparison fair and shows that a more informed heuristic can reduce the search effort without changing the optimal result.

Memory consumption remains an important consideration when using A*. The algorithm stores information about discovered and expanded states to manage the search and reconstruct the solution path. For the 8-puzzle, the reachable state space is limited to 181,440 states, making this manageable. For larger puzzles, memory requirements can become a significant limitation. An alternative such as IDA* could be considered because it uses less memory.

Experience

Before this task, we knew A* mostly from the lecture. Implementing it ourselves showed us how much the heuristic matters. We expected Manhattan to perform better, but seeing it expand approximately 16 times fewer nodes made the importance of a more informed heuristic much clearer.

We also learned that a correct result is not enough to trust an implementation. Checking that both heuristics return the same solution length turned out to be a simple but reliable test. Using a fixed random seed helped as well, because we could repeat a run and compare the results after every change.

Finally, the measurements taught us to look beyond average values. Some puzzles required far more effort than others, so the standard deviation was relatively high compared to the mean. This helped us understand why evaluating an algorithm on multiple test cases is important.

Possible improvements

Improvement	Expected effect
Linear conflict added to Manhattan	Stronger admissible heuristic, fewer expanded nodes
Pattern database heuristic	Much stronger estimate, at the cost of precomputation and memory
IDA* instead of A*	Lower memory consumption, especially for larger puzzles
Incremental heuristic update	Only the moved tile needs to be considered, reducing repeated calculations
State packed into one integer	Less memory per stored state and potentially faster hashing
Repeated timing per puzzle	More reliable execution time measurements with less noise
Generalisation to N×N boards	Supports larger puzzle sizes, the solvability rule must account for the blank row for even-width boards